

DarkWave Task Manager: Integrating an LLM Into a Task Management System

Exploring the integration of large language models to enhance productivity in task management systems.

DarkWave Task Manager: Integrating an LLM Into a Task Management System

This presentation, led by Douglas Fulcher for the Applied Data Science Capstone course at Bellevue University, focuses on the DarkWave Task Manager. It is an AI-powered interface designed to allow users to create and retrieve tasks in Todoist using natural language. The project aims to demonstrate how large language models can streamline productivity by minimizing the friction caused by traditional task management processes. Unlike typical DSC 680 projects, which primarily emphasize modeling and prediction, this initiative emphasizes systems architecture, validation, tooling, and the secure integration of LLMs into practical applications.



Business Problem & Motivation

Administrative Burden on Productivity

People often spend excessive time on administrative tasks essential for capturing tasks in productivity applications. This includes navigating through multiple steps like selecting projects, filling in fields, setting due dates, and prioritizing tasks, all of which can disrupt focus and efficiency.



Key Research Questions

The research questions focused on the ability of a language model to understand natural language task requests and identify effective prompting strategies. These strategies are crucial for extracting structured information such as task titles, due dates, and priorities from user inputs.



Importance of Validation

Validation of LLM outputs is essential to ensure they meet the strict requirements of systems like the ToDoist API. This involves checking the structure and accuracy of the data before it is sent to prevent errors and maintain system integrity.



Containerization and Security

Understanding how to securely containerize and deploy the application is vital for real-world use, ensuring that the integration of AI into productivity systems does not compromise security or performance.



Administrative Burden on Productivity

People often spend excessive time on administrative tasks essential for capturing tasks in productivity applications. This includes navigating through multiple steps like selecting projects, filling in fields, setting due dates, and prioritizing tasks, all of which can disrupt focus and efficiency.



Key Research Questions

The research questions focused on the ability of a language model to understand natural language task requests and identify effective prompting strategies. These strategies are crucial for extracting structured information such as task titles, due dates, and priorities from user inputs.



Importance of Validation

Validation of LLM outputs is essential to ensure they meet the strict requirements of systems like the Todoist API. This involves checking the structure and accuracy of the data before it is sent to prevent errors and maintain system integrity.



Containerization and Security

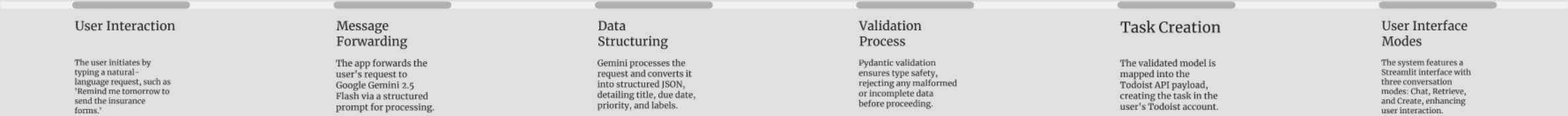
Understanding how to securely containerize and deploy the application is vital for real-world use, ensuring that the integration of AI into productivity systems does not compromise security or performance.

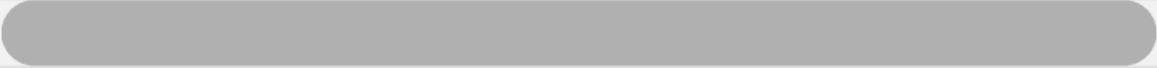




System Overview & Architecture

Understanding the key components and workflows of the DarkWave Task Manager system.





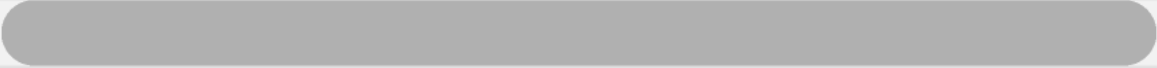
User Interaction

The user initiates by
typing a natural-
language request, such as
'Remind me tomorrow to
send the insurance
forms.'



Message Forwarding

The app forwards the
user's request to
Google Gemini 2.5
Flash via a structured
prompt for processing.



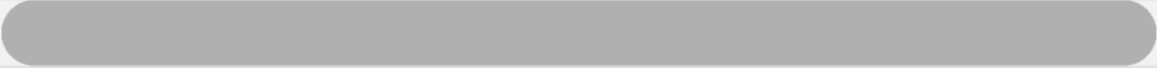
Data Structuring

Gemini processes the request and converts it into structured JSON, detailing title, due date, priority, and labels.



Validation Process

Pydantic validation ensures type safety, rejecting any malformed or incomplete data before proceeding.



Task Creation

The validated model is mapped into the Todoist API payload, creating the task in the user's Todoist account.



User Interface Modes

The system features a Streamlit interface with three conversation modes: Chat, Retrieve, and Create, enhancing user interaction.

LLM Service Layer

The LLM Service Layer is responsible for handling prompting, setting model parameters, and mapping the responses from the language model into structured Pydantic models. This layer ensures effective communication between the natural language input and the structured output required by the system.

Validation Layer

The Validation Layer utilizes Pydantic to enforce strict rules, preventing the LLM from sending malformed data. It ensures that any inputs or outputs adhere to the expected formats, enhancing reliability and reducing errors.

Centralized Configuration

The project utilizes pyproject.toml for centralized configuration and dependency management through uv, ensuring a clean and efficient development workflow. This modern approach streamlines project setup and maintenance.

Todoist API Client

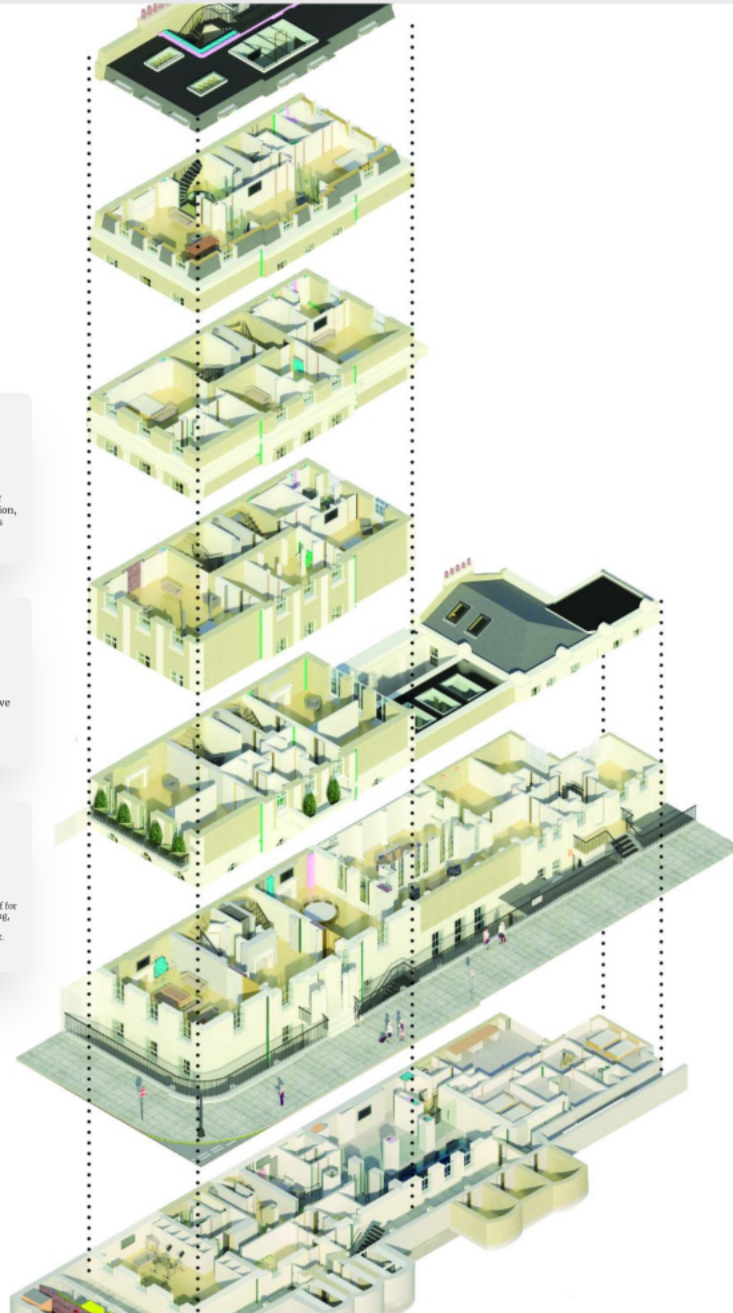
The Todoist API Client acts as a clean, typed wrapper for interacting with the Todoist API, managing authentication, validation, and task creation. This abstraction simplifies API calls and enhances code maintainability.

UI Layer with Streamlit

The UI Layer, built on Streamlit, provides an interactive and user-friendly chat experience. It enables users to engage with the system effortlessly, making task management intuitive and accessible.

Quality Control Tools

Quality control is implemented using various tools, including ruff for linting, mypy for static type checking, bandit for security scanning, and pre-commit hooks for maintaining code quality. The use of Podman ensures consistency in containerization and deployment.



Code Structure & Development Framework

LLM Service Layer

The LLM Service Layer is responsible for handling prompting, setting model parameters, and mapping the responses from the language model into structured Pydantic models. This layer ensures effective communication between the natural language input and the structured output required by the system.

Todoist API Client

The Todoist API Client acts as a clean, typed wrapper for interacting with the Todoist API, managing authentication, validation, and task creation. This abstraction simplifies API calls and enhances code maintainability.

Validation Layer

The Validation Layer utilizes Pydantic to enforce strict rules, preventing the LLM from sending malformed data. It ensures that any inputs or outputs adhere to the expected formats, enhancing reliability and reducing errors.

UI Layer with Streamlit

The UI Layer, built on Streamlit, provides an interactive and user-friendly chat experience. It enables users to engage with the system effortlessly, making task management intuitive and accessible.

Centralized Configuration

The project utilizes `pyproject.toml` for centralized configuration and dependency management through `uv`, ensuring a clean and efficient development workflow. This modern approach streamlines project setup and maintenance.

Quality Control Tools

Quality control is implemented using various tools, including ruff for linting, mypy for static type checking, bandit for security scanning, and pre-commit hooks for maintaining code quality. The use of Podman ensures consistency in containerization and deployment.

Create Mode: Natural Language Interaction

In Create Mode, the app allows the user to quickly create tasks in Todoist using natural language prompts using prompts like "Remind me to schedule a dentist appointment by Friday."



Retrieve Mode: Task Access Made Easy

Retrieve Mode allows users to ask specific questions like 'What tasks do I have?' or 'Show me everything due today.' The system connects to the Todoist API to fetch relevant tasks, presenting them with clickable links for easy access.



What Worked Well & Key Findings

LLMs Excel at Task Interpretation

The LLM effectively interpreted natural language tasks, with Gemini demonstrating reliable parsing of user requests into structured payloads, enhancing user interaction and task management efficiency.



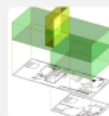
Importance of Validation

Validation proved to be a crucial aspect, as it ensured that the LLM did not produce incomplete or incorrect task details. Pydantic effectively captured nearly all edge cases, safeguarding data integrity before API interactions.



Architecture Over Features

Separation of concerns in architecture was key, maintaining distinct layers for LLM logic, API calls, and UI code facilitated easier testing, extension, and debugging processes, leading to a more robust system.



Impact of Tooling

Utilizing modern tooling, including uv, ruff, mypy, bandit, and Pylint, established a production-like workflow that streamlined development processes and enhanced overall efficiency.



Consistency Through Containers

Running the application within containers provided a consistent execution environment, crucial for ensuring reliability before deployment and minimizing unexpected behavior during use.



LLMs Excel at Task Interpretation

The LLM effectively interpreted natural language tasks, with Gemini demonstrating reliable parsing of user requests into structured payloads, enhancing user interaction and task management efficiency.



Importance of Validation

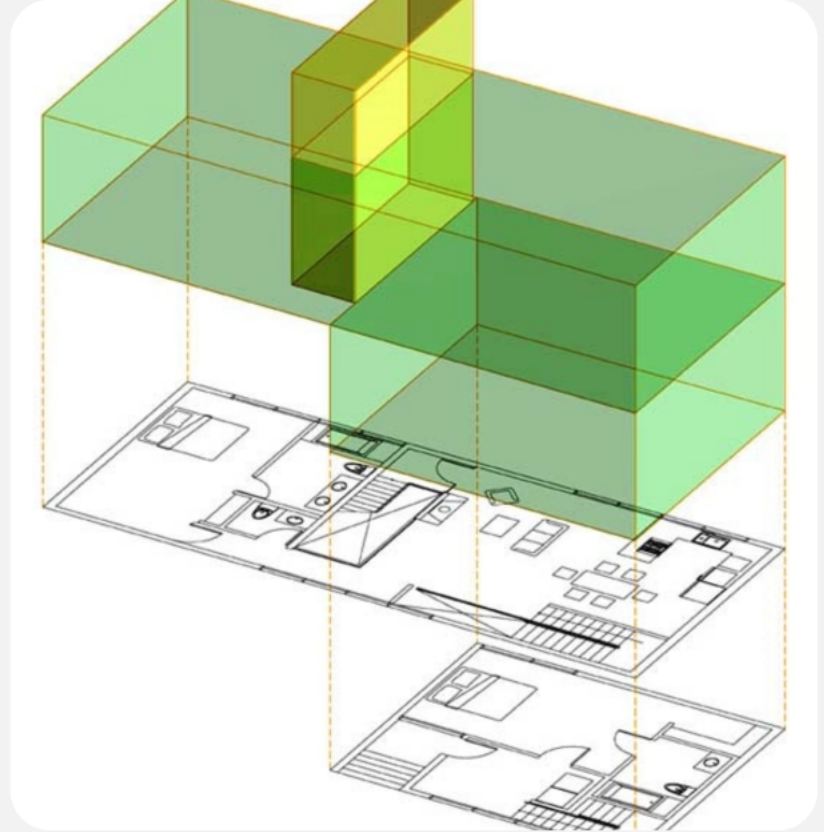
Validation proved to be a crucial aspect, as it ensured that the LLM did not produce incomplete or incorrect task details. Pydantic effectively captured nearly all edge cases, safeguarding data integrity before API interactions.



```
    p, '\w+-\d{6}');  
    n(@(c) ~isempty(c), nscans;  
    (scanIdx);  
  
    Ids{:}});  
  
    , string(), 'Id', string(),  
    nScans, 1), 'ImageLabel', cell(nScans, 1),  
    else, 'Pan', cell(nScans, 1),  
    nScans, 1), 'Range', cell(nScans, 1),  
    ar')  
    progressBar(nScans, 'Progress', 0,  
  
    s)  
    (scanIds{i})  
    scanIds{i}(j);
```

Architecture Over Features

Separation of concerns in architecture was key; maintaining distinct layers for LLM logic, API calls, and UI code facilitated easier testing, extension, and debugging processes, leading to a more robust system.



Impact of Tooling

Utilizing modern tooling, including uv, ruff, mypy, bandit, and Podman, established a production-like workflow that streamlined development processes and enhanced overall efficiency.



Consistency Through Containers

Running the application within containers provided a consistent execution environment, crucial for ensuring reliability before deployment and minimizing unexpected behavior during use.



Limitations & Challenges

This project, being exploratory in nature, presents notable limitations. Currently, only task creation and retrieval functionalities are operational, with no editing, completion, or filtering capabilities. The error handling mechanisms are basic and require further development for practical applications. Additionally, the absence of multi-user support, access control, and detailed authentication measures limits its use. Although the security protocols are sufficient for prototyping, real-world deployment would demand enhancements such as credential rotation, secret management, and audit logging. The current prompting strategies are effective but not fully optimized, leading to occasional confusion from user input. These limitations underline the project's focus on foundational AI integration patterns rather than on delivering a comprehensive SaaS product.





Future Work & Real-World Applications

Potential enhancements and broader applications for the DarkWave Task Manager project.

Upcoming Enhancements

Future work includes adding features such as task modifying, completing, and snoozing, along with smarter LLM flows for workload summaries and priority suggestions.

Integration with Other Tools

Plans to integrate with calendar and email APIs to enhance task management capabilities and streamline user workflows.

Deployment Improvements

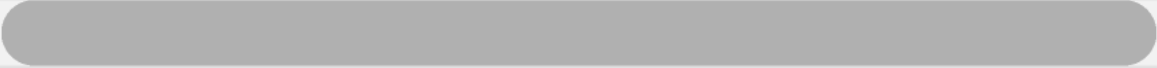
Implementing a CI/CD pipeline will ensure consistent and efficient deployment processes for updates and enhancements.

Multi-User Support

Extending the architecture to securely support multiple users, enhancing collaboration and task management capabilities.

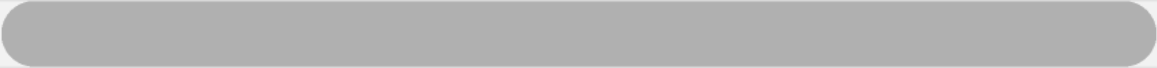
Broader Applications

The project demonstrates reusable patterns for CRM task automation, IT helpdesk assistants, customer-service workflows, and personal assistants built on enterprise data.



Upcoming Enhancements

Future work includes adding features such as task modifying, completing, and snoozing, along with smarter LLM flows for workload summaries and priority suggestions.



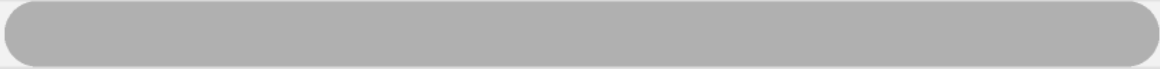
Integration with Other Tools

Plans to integrate with
calendar and email APIs
to enhance task
management capabilities
and streamline user
workflows.



Deployment Improvements

Implementing a CI/CD pipeline will ensure consistent and efficient deployment processes for updates and enhancements.



Multi-User Support

Extending the
architecture to securely
support multiple users,
enhancing collaboration
and task management
capabilities.



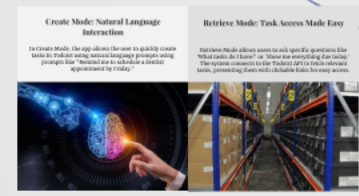
Broader Applications

The project demonstrates reusable patterns for CRM task automation, IT helpdesk assistants, customer-service workflows, and personal assistants built on enterprise data.

Conclusion

The DarkWave Task Manager successfully integrates an LLM to convert natural language into structured Todoist tasks. Rather than focusing on a wide array of features, the project prioritized robust architecture, thorough validation, modern development tools, and secure LLM integration. This endeavor enhanced my expertise in contemporary Python methodologies, containerization, type safety, prompt engineering, and practical applications of LLMs. Although the system is designed with specific limitations, it establishes a strong foundation for future enhancements and scalability.





DarkWave Task Manager: Integrating an LLM Into a Task Management System

Exploring the integration of large language models to enhance productivity in task management systems.